



Introduction to trees

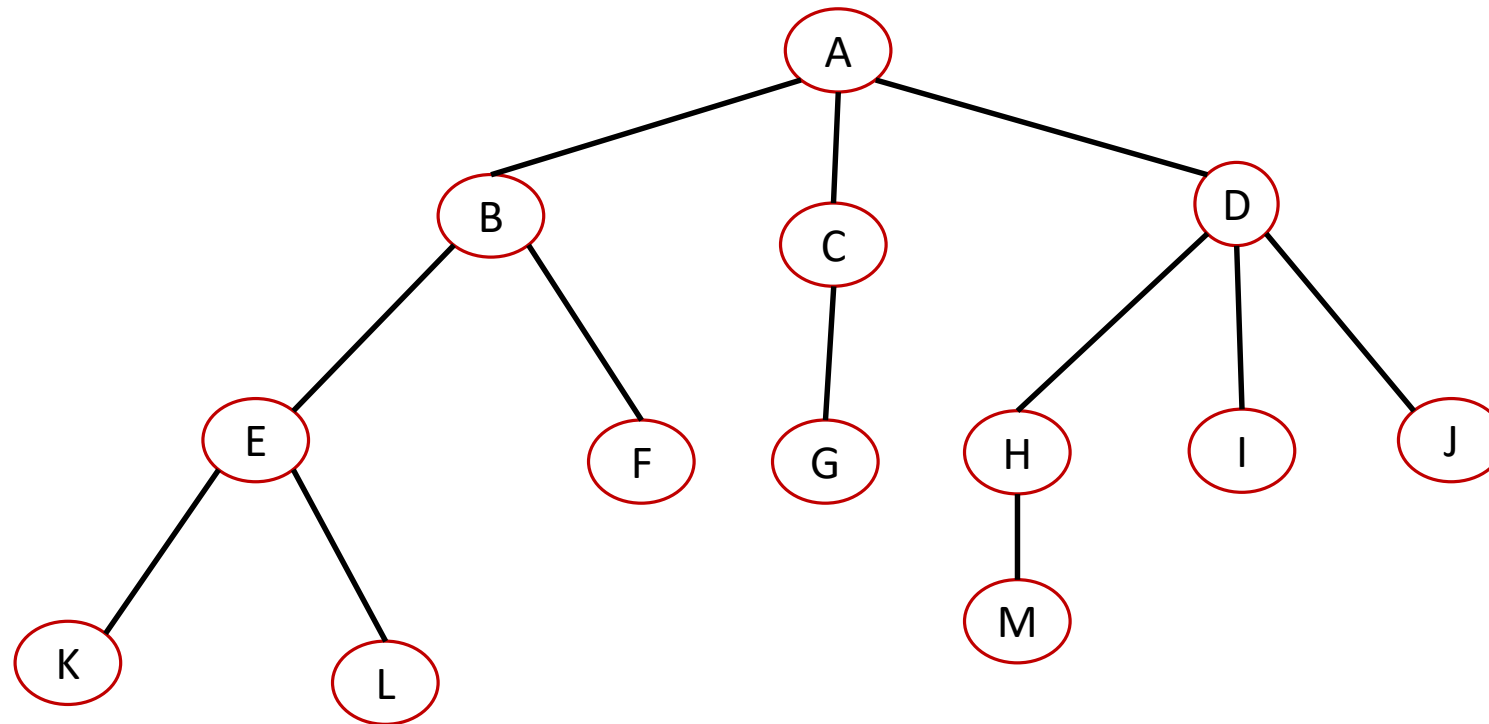
MODULE 4 PART 2

Representation of Trees

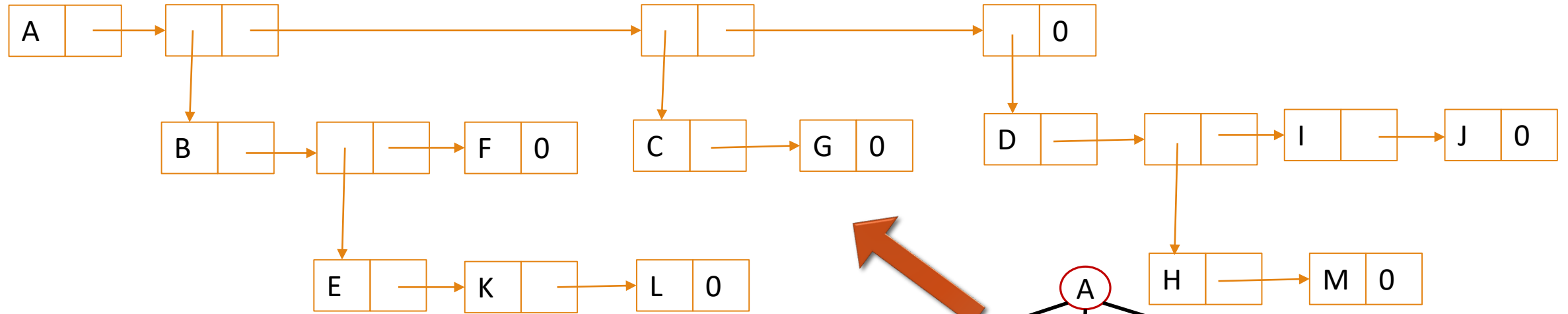
List Representation: The list representation for the following tree can be as follows,

(A(B (E (K, L), F), C (G), D(H(M), I,J)))

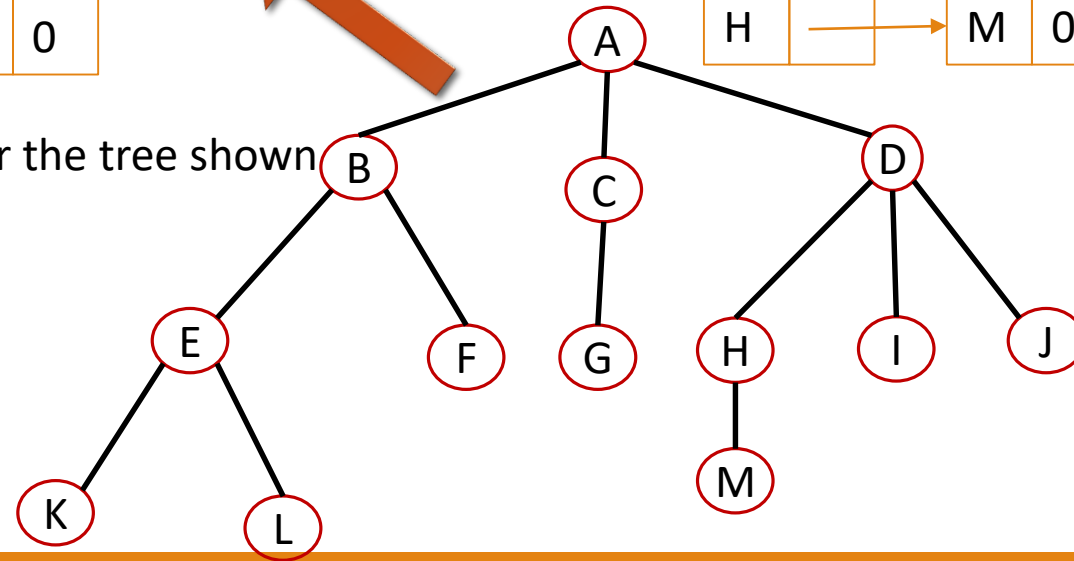
The information in the root node comes first, followed by a list of the subtrees of that node.



List Representation cntd..



The above figure shows memory representation for the tree shown



List Representation cntd..

For several applications, it is desirable to have a representation that is specialized to trees. One possibility is to represent each tree node by a memory node that has fields for the data and pointers to the tree node's children.

For a tree of degree k , the following node structure can be used.

DATA	CHILD1	CHILD2		CHILDK
------	--------	--------	------	--------

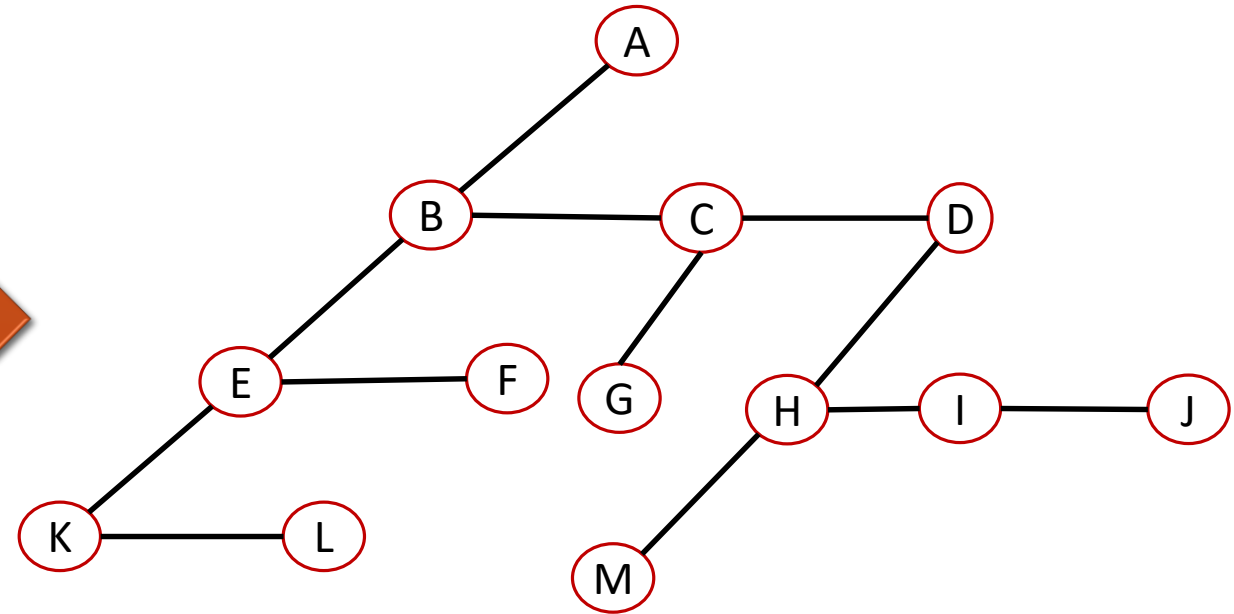
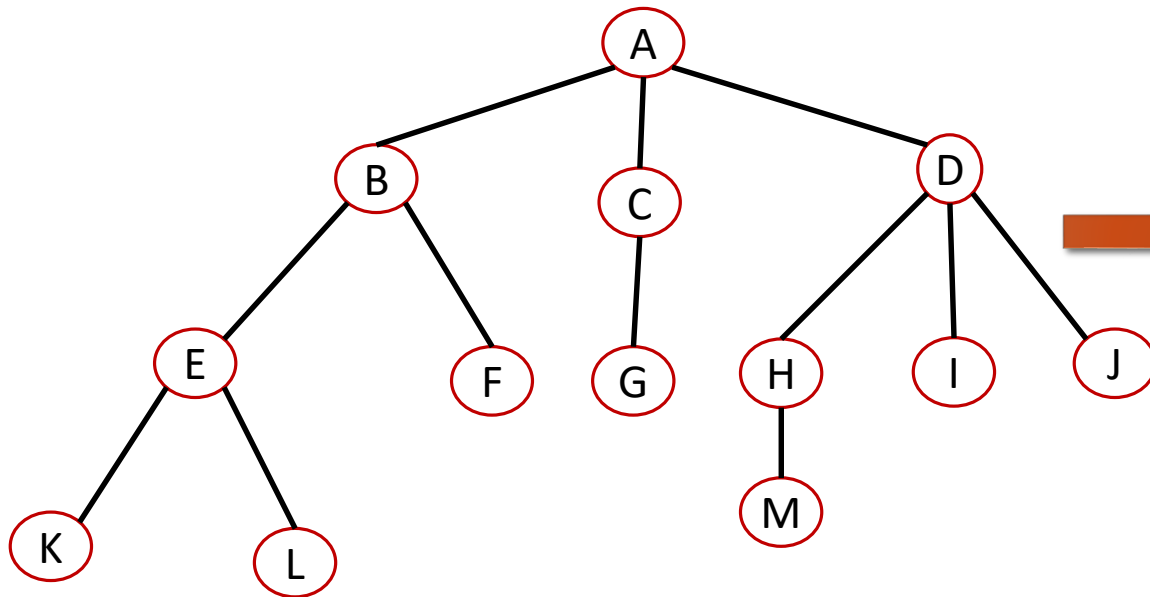
Possible node structure for a tree of degree k

Left Child-Right Sibling Representation

- The *left child* field of each node points to its leftmost child (if any), and the *right sibling* field points to its closest right sibling.

DATA	
Left child	Right sibling

Left child-right sibling node structure

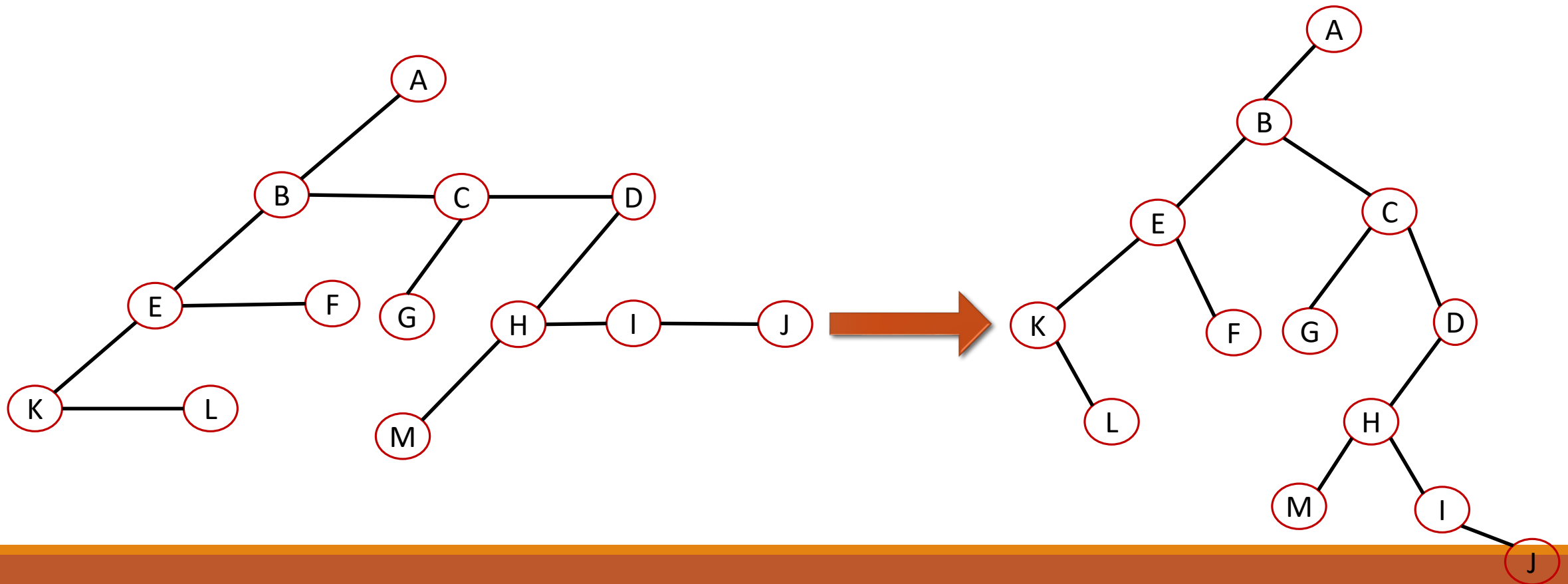


Left child-right sibling representation

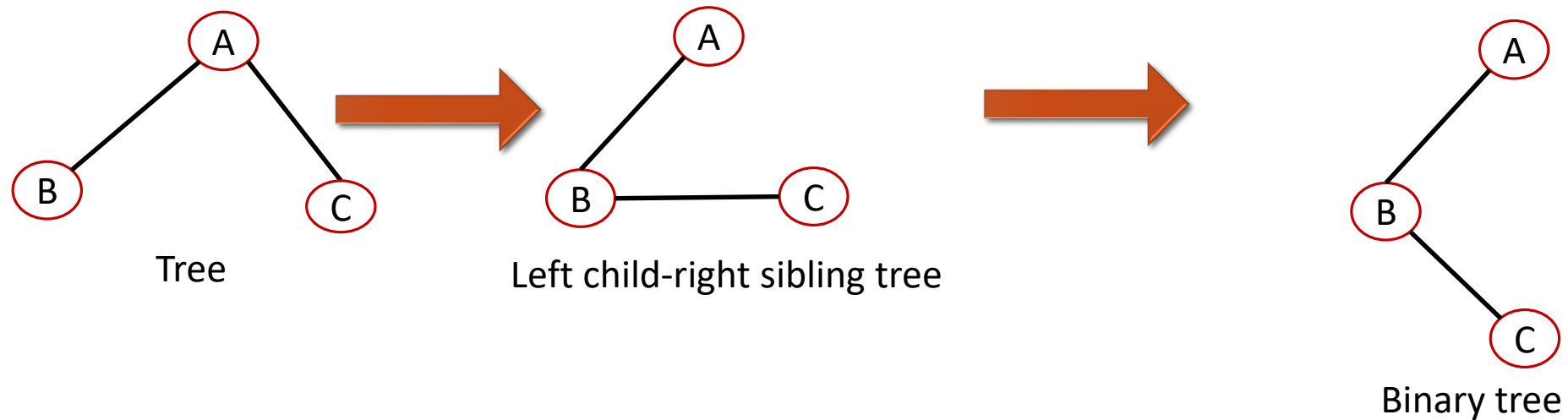
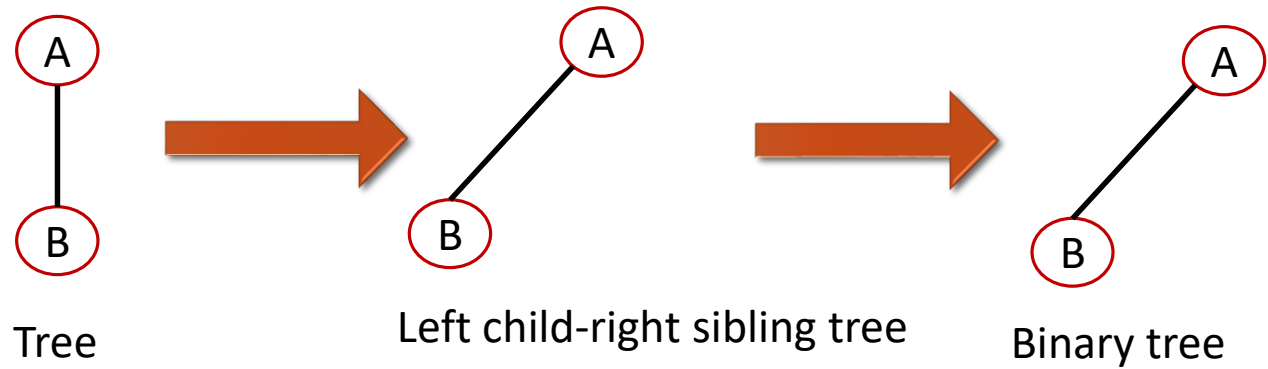
Representation as a Degree-Two Tree

To obtain the degree-two tree representation of a tree, we simply rotate the right-sibling pointers in a left child-right sibling tree clockwise 45 degrees. This gives us the degree-two tree displayed below. The right child of root node is empty.

Left child-right child (or degree two) trees are also called as *binary trees*.



Degree-Two Tree Representations examples



BINARY TREES

A binary tree is a finite set of nodes that is either empty or consists of a root and two disjoint binary trees called the left subtree and right subtree.

Tree	Binary Tree
There is no tree having zero nodes	There is an empty binary tree
There is no distinction between order of the children. If viewed as a tree, the two trees shown in Fig. 4 are same.	We distinguish between order of the children Hence, the two binary trees shown in Fig. 4 are different. Since, the first binary tree has an empty right sub tree and the second has an empty left subtree.

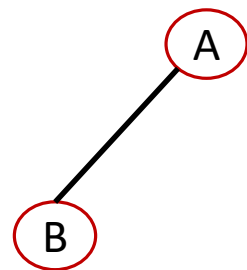
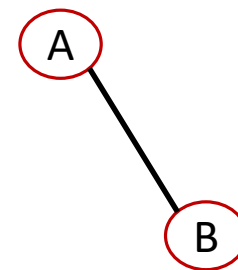
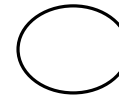
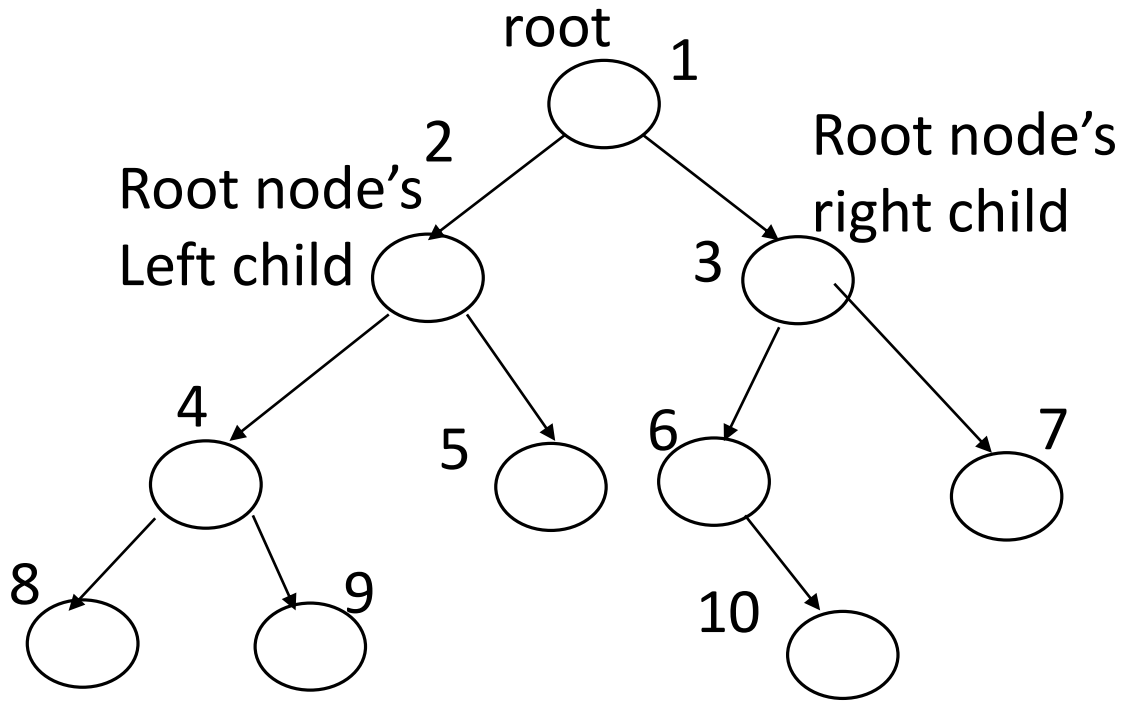


Fig 4

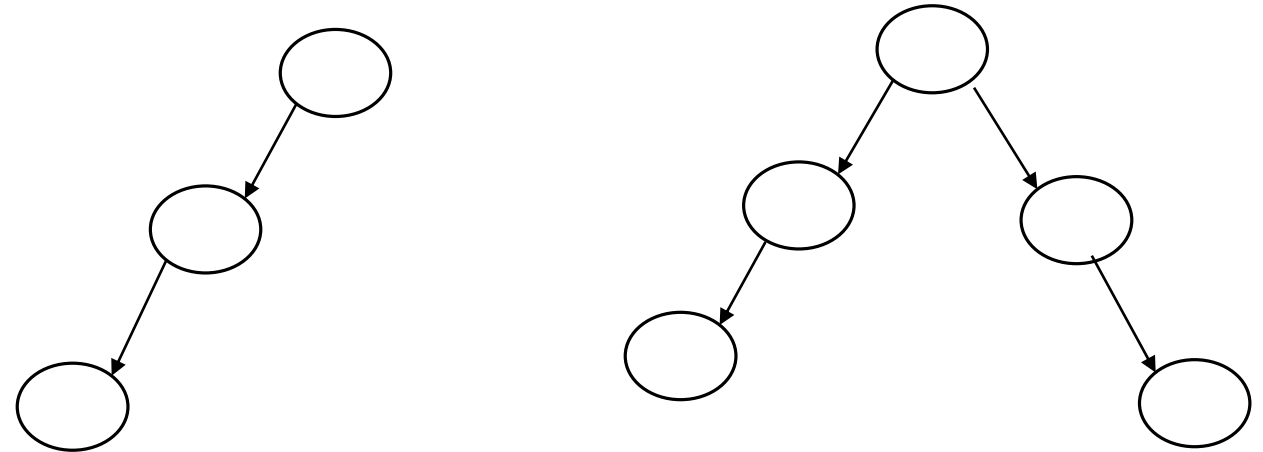




Binary tree cntd..



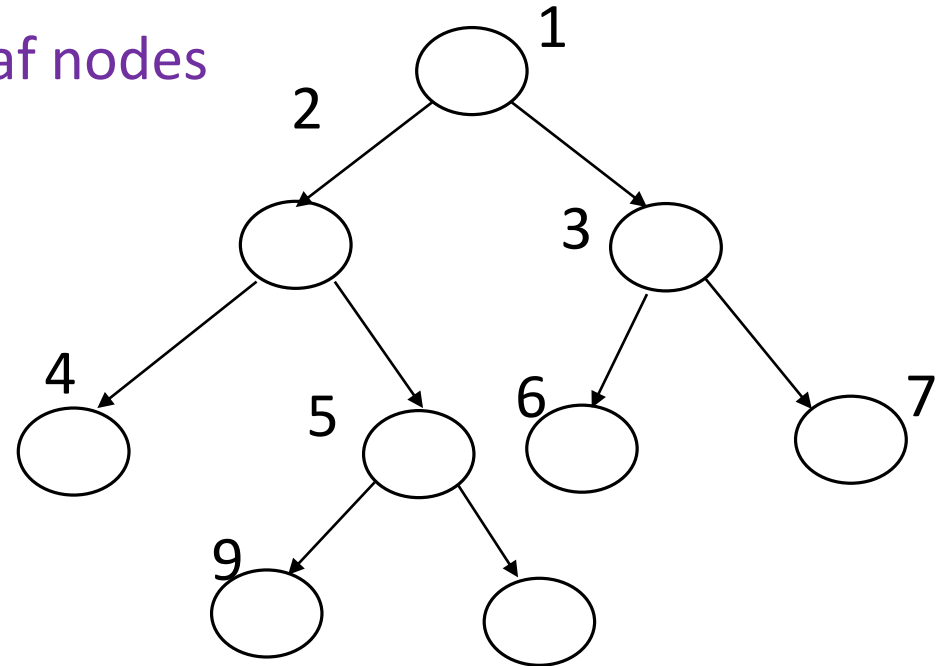
Binary tree



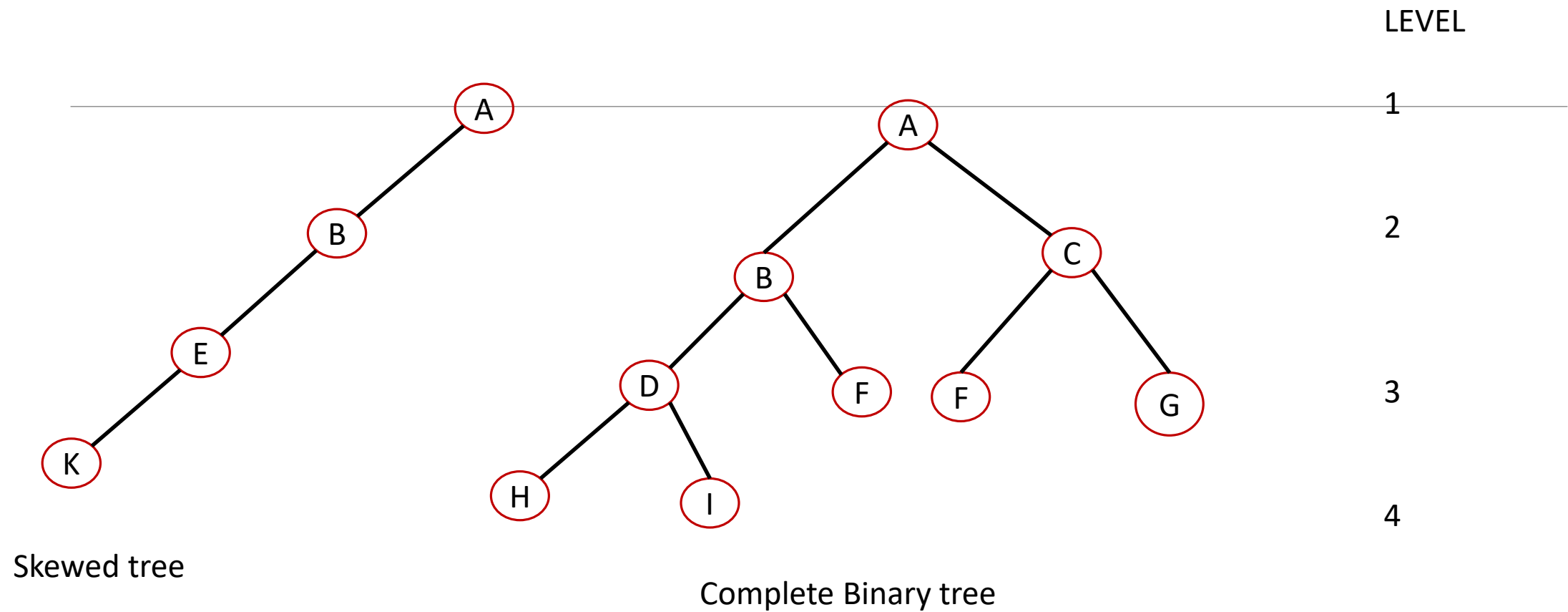
Binary tree examples

Strict binary tree or proper binary tree

- A binary tree is called **strict binary tree** or **proper binary tree** if it has 0 or 2 children.
- A node with zero child are called **leaf nodes**

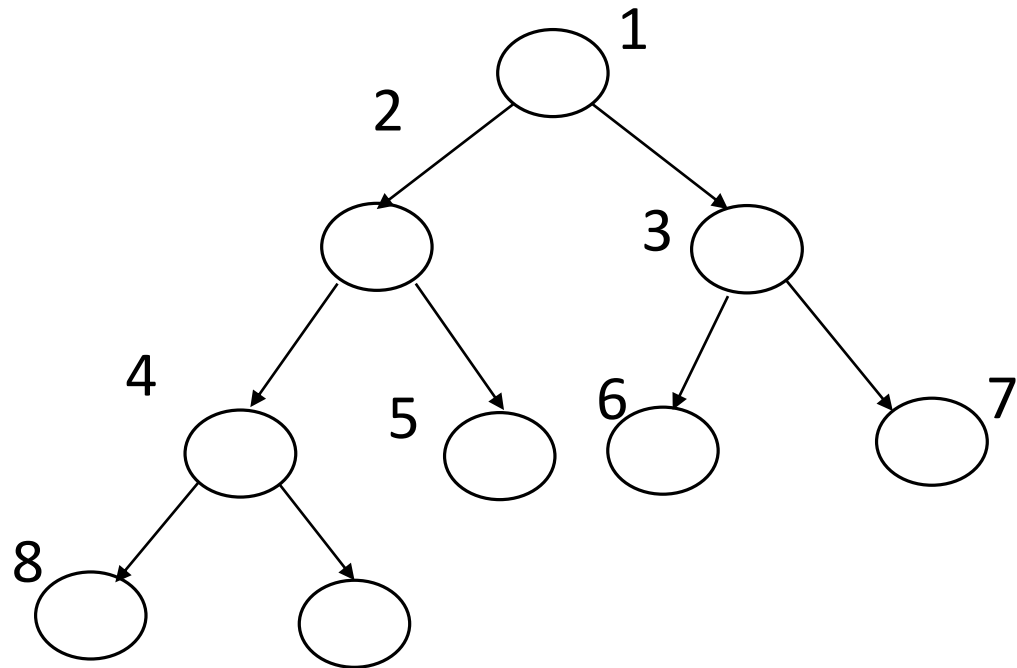


SPECIAL KINDS OF BINARY TREES



Complete binary tree

- **Complete binary tree:** All levels except possibly the last are completely filled and all nodes are as left as possible.



Properties of Binary Trees

The maximum number of nodes on level i of a binary tree is 2^i , $i \geq 0$.

Number of nodes at level 0 is 2^0

Number of nodes at level 1 is 2^1

Number of nodes at level 2 is 2^2

...

Number of nodes at level h is 2^h

The maximum number of nodes (n) of height (h) = $2^0 + 2^1 + 2^2 + \dots + 2^h = 2^{h+1} - 1$.

The minimum number of nodes (n) of height (h) = $h + 1$

Maximum height $h = n - 1$

Minimum height $h = \log_2(N + 1) - 1$

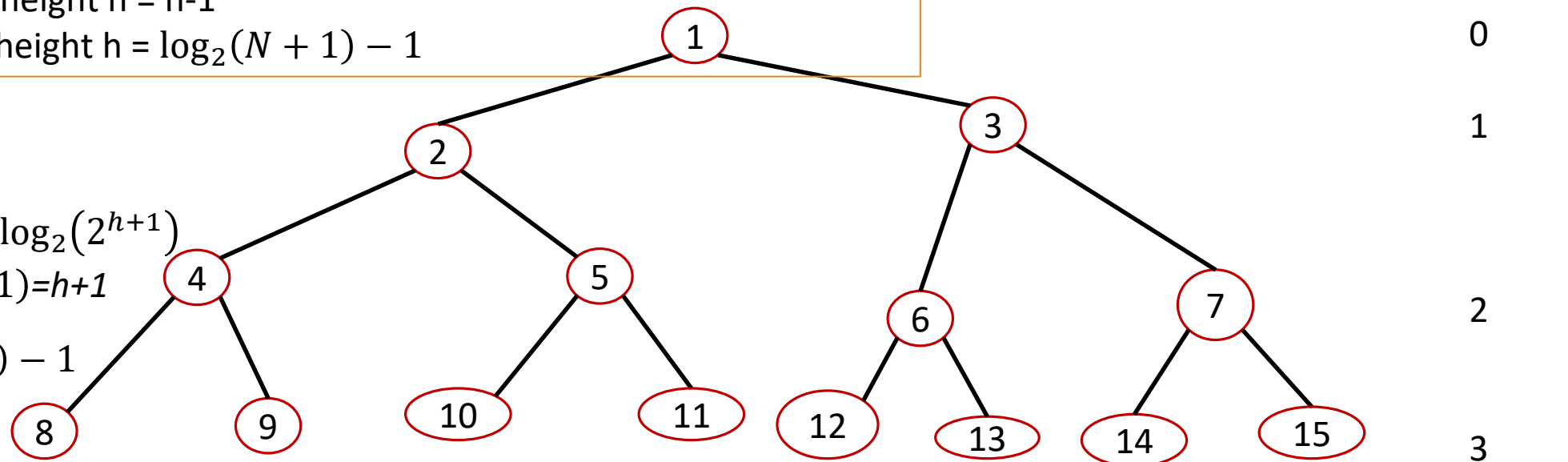
$$N = 2^{h+1} - 1$$

$$N + 1 = 2^{h+1}$$

$$\log(N + 1) = \log_2(2^{h+1})$$

$$\log_2(N + 1) = h + 1$$

$$h = \log_2(N + 1) - 1$$



Further, I have to add more properties here .

Binary tree representation

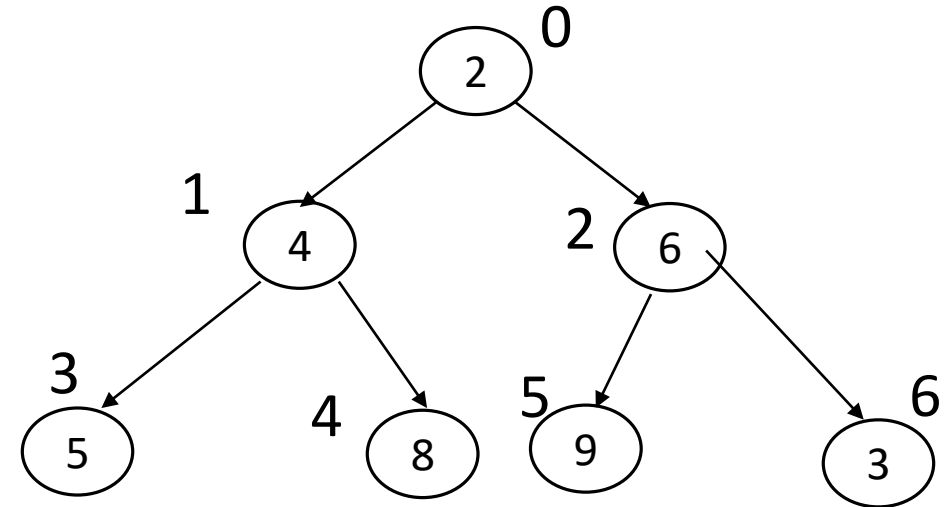
We can implement binary tree using,

arrays

For node at index i , Complete binary tree

left-child-index = $2i + 1$

right-child-index = $2i + 2$



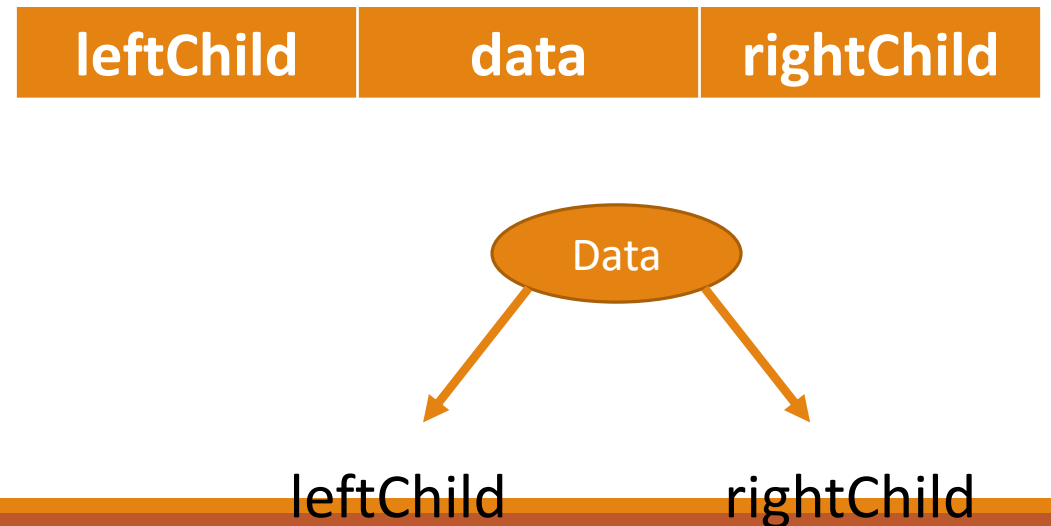
0	1	2	3	4	5	6
2	4	6	5	8	9	3

Tree representation linked list examples

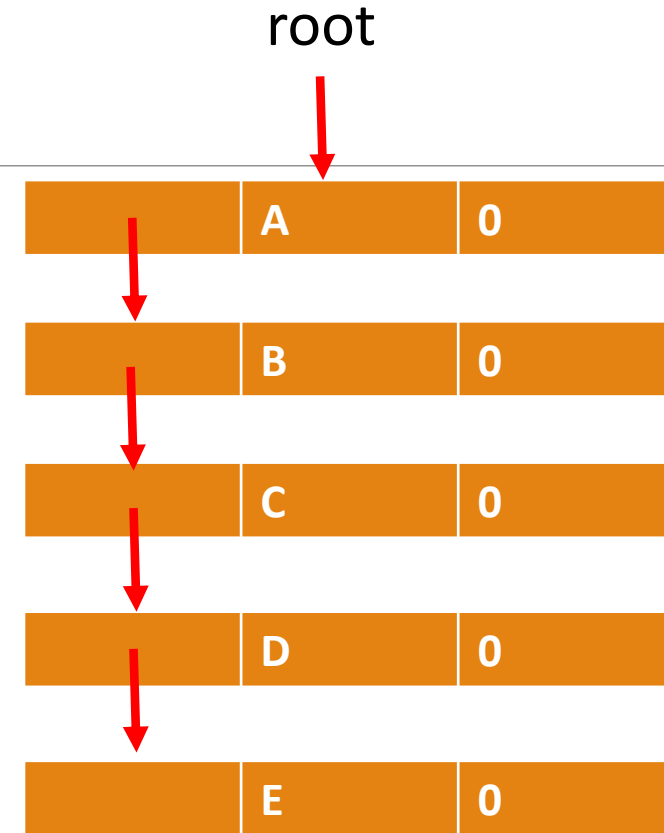
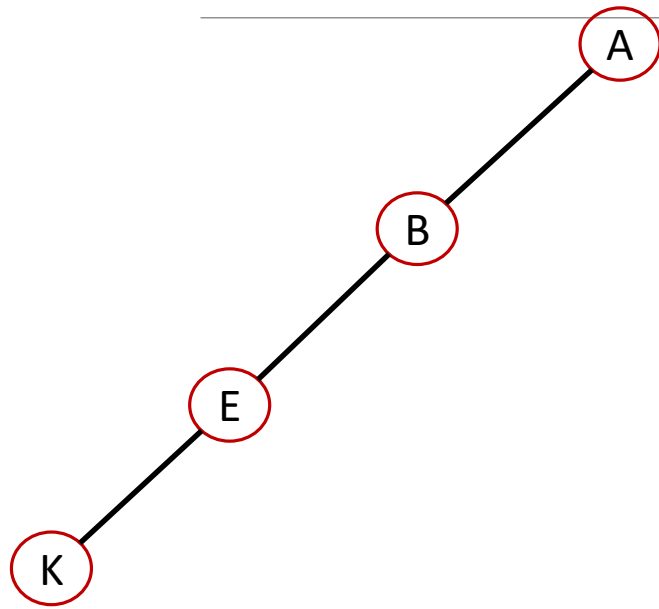
Array representation is wasteful of many other binary trees. In addition, the representation suffers from the general inadequacies of sequential representations. Insertion and deletion of nodes from the middle of a tree require the movement of potentially many nodes to reflect the change in level number of these nodes. These problems can be overcome easily through the use of a linked representation.

Each node has three fields: leftChild, data and rightChild. It can be defined in C as:

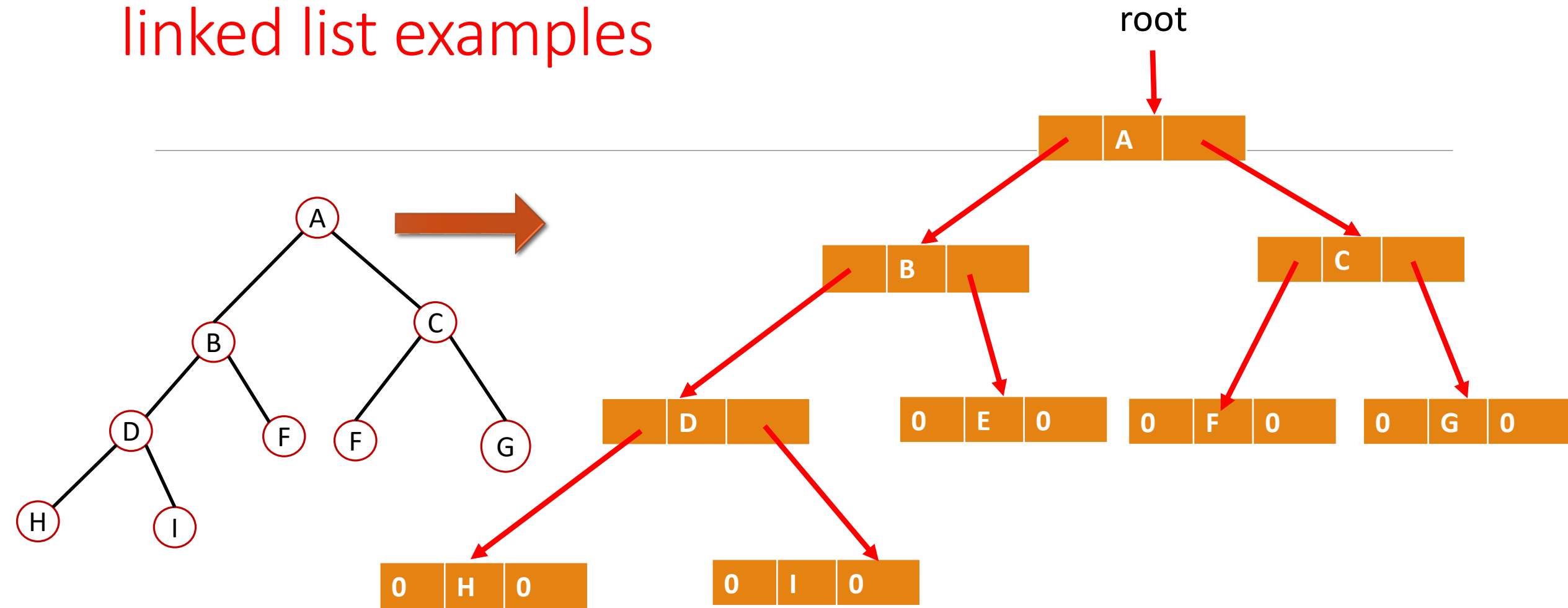
```
typedef struct node *treePointer;  
typedef struct {  
    int data;  
    treePointer leftChild, rightChild;  
} node;
```



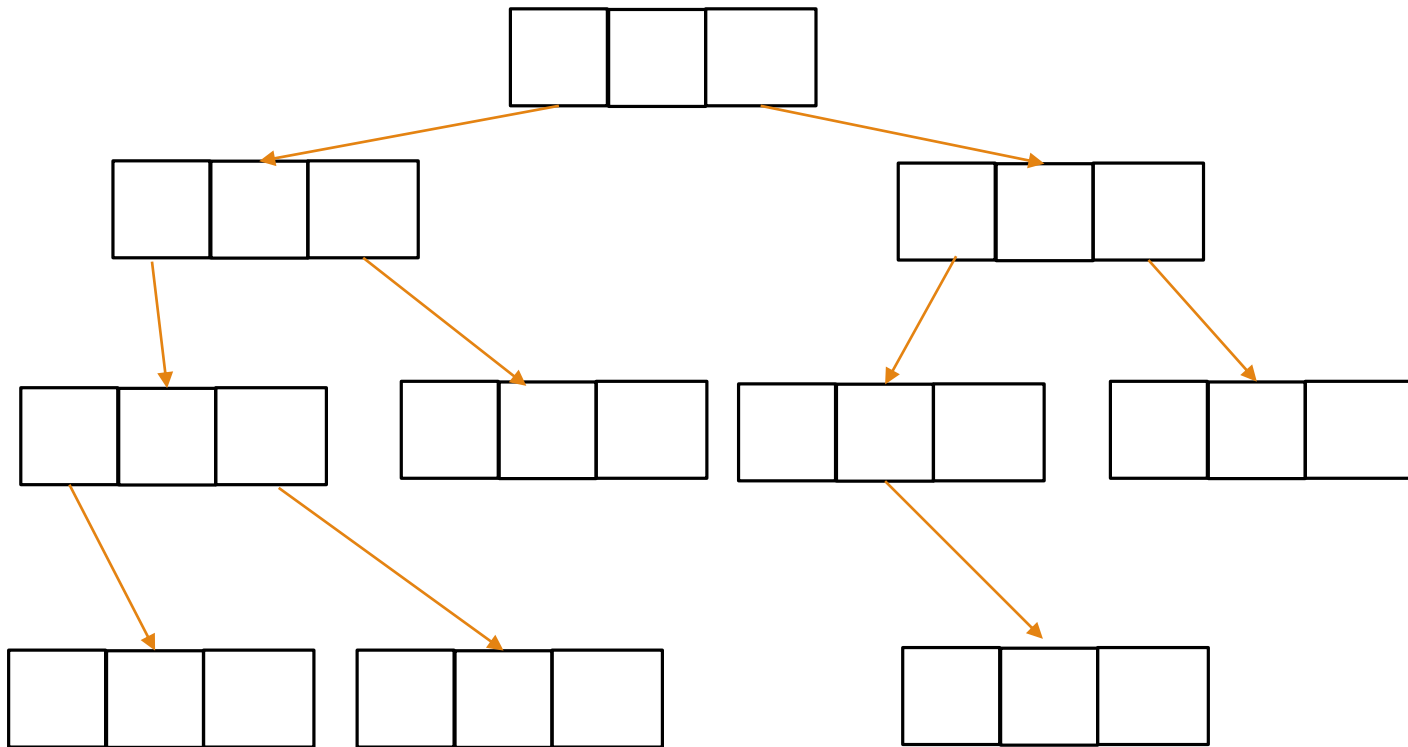
Tree representation using linked list examples



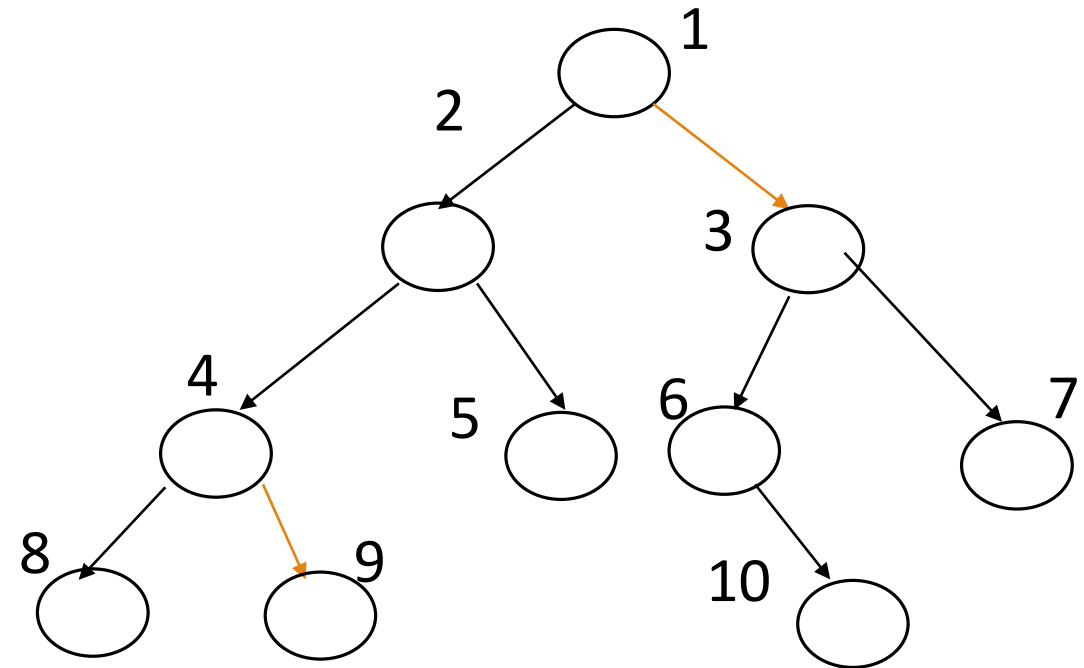
Tree representation using linked list examples



Binary tree



Linked representation of Binary tree



Binary tree

Each node can have at most 2 children